Processingをはじめよう

第7章

動き　その２

目次

- フレームレート
- スピードと方向
- 2点間の移動
- 乱数
- タイマー
- 円運動

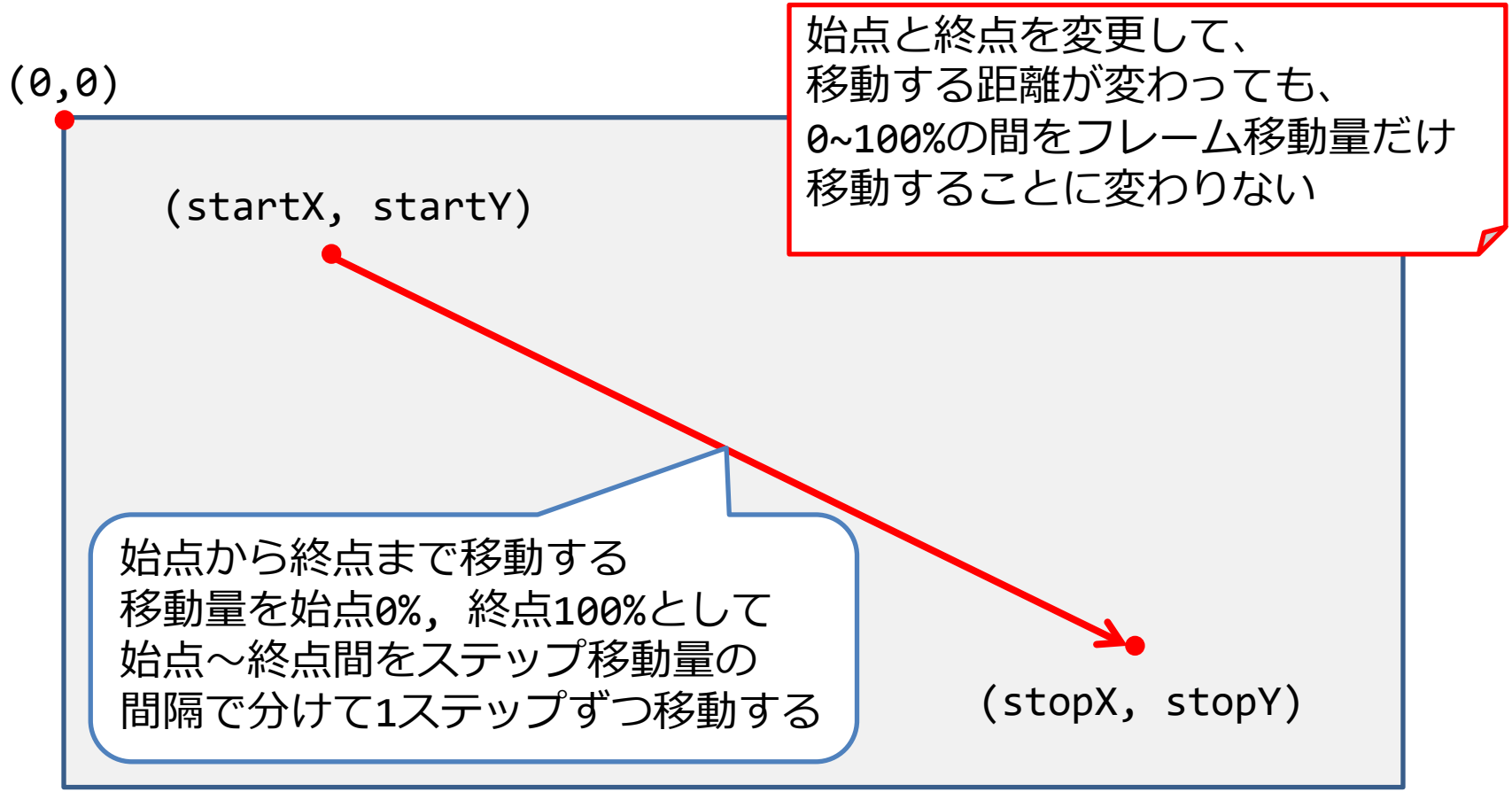
今回はここまで

- 移動、回転、拡大、縮小

2点間の移動

- Example 7-6 (EX_08_06)
  - 始点(startX, startY)から
    終点(stopX, stopY)まで移動する
  - 座標更新の計算方法は後述
  - 始点と終点を変更しても動作する、変更して確認
    - 始点(0,0), 終点(width, height)
    - 始点(0, height) 終点(0, width)

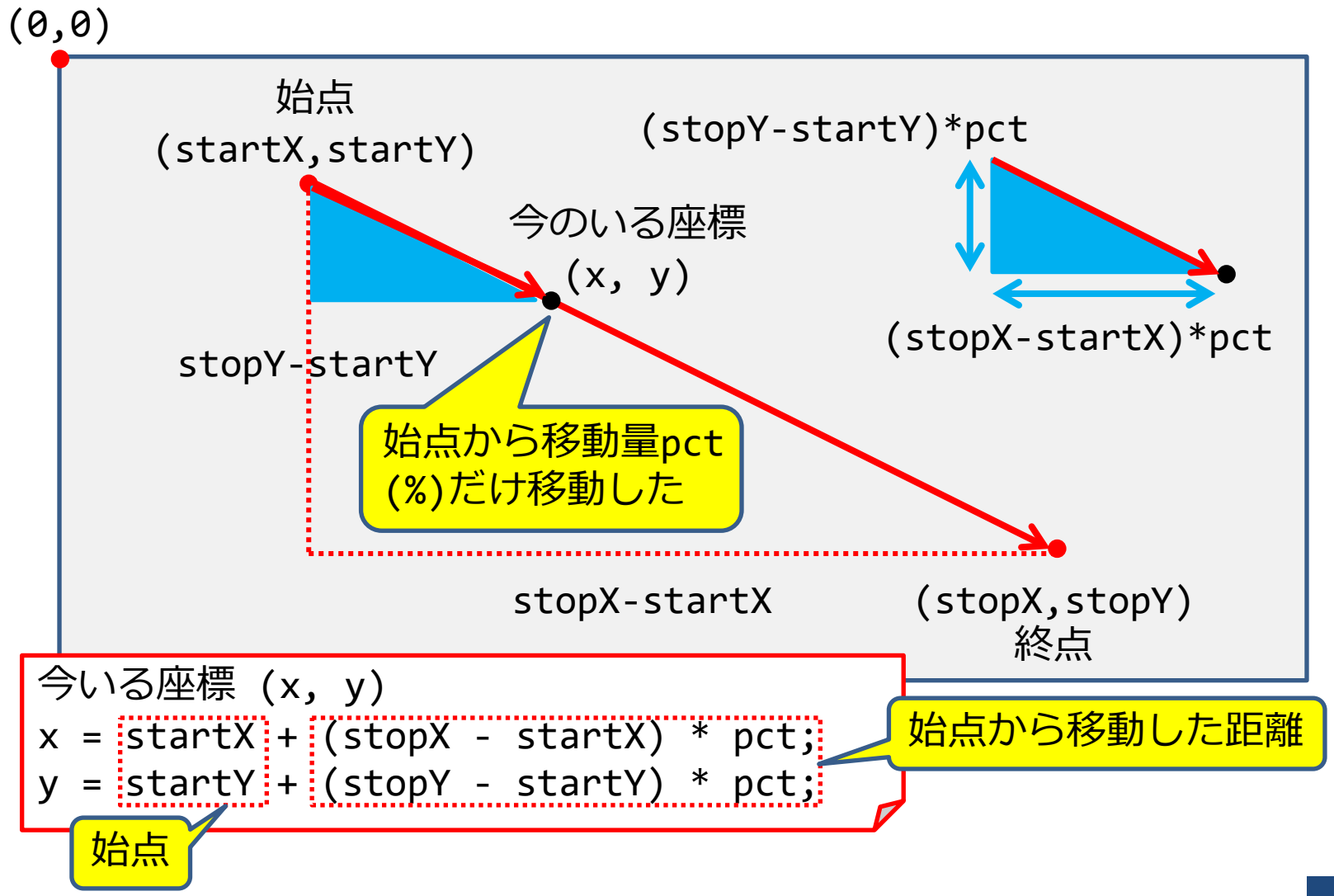
2点間の移動


(0,0)
始点と終点を変更して、
移動する距離が変わっても、
0~100%の間をフレーム移動量だけ
移動することに変わりない
(startX, startY)
始点から終点まで移動する
移動量を始点0%，終点100%として
始点～終点間をステップ移動量の
間隔で分けて1ステップずつ移動する
(stopX, stopY)

Example 7-6

```
int startX = 20;      // 始点のx座標
int stopX = 160;      // 終点のx座標
int startY = 30;      // 始点のy座標
int stopY = 80;       // 終点のy座標
float x = startX;     // 現在のx座標
float y = startY;     // 現在のy座標
float step = 0.005;   // 1フレームの移動量(0.0 ~ 1.0で設定する)
float pct = 0.0;      // 移動量(0.0 ~ 1.0の範囲の値をとる)

void setup() {
  size(240, 120);
}

void draw() {
  background(0);
  if (pct < 1.0) {                          // 移動量が100%より小さい場合（終点まで移動していない）
    x = startX + ((stopX-startX) * pct);    // 現在のx座標を更新
    y = startY + ((stopY-startY) * pct);    // 現在のy座標を更新
    pct += step;                            // 移動量を更新
  }
  ellipse(x, y, 20, 20);
}
```

2点間の移動距離を計算



今いる座標 (x, y)

```
x = startX + (stopX - startX) * pct;
y = startY + (stopY - startY) * pct;
```

乱数 Example 7-7

- Example 7-7 (EX_08_07) 乱数の生成
  - random()命令 P205
    random(low, high)
    low以上 high未満の乱数を生成する

```
void draw() {
  float r = random(0, mouseX);        // 0以上mouseX未満の乱数生成
  println(r);                         // 乱数を表示
}
```

乱数

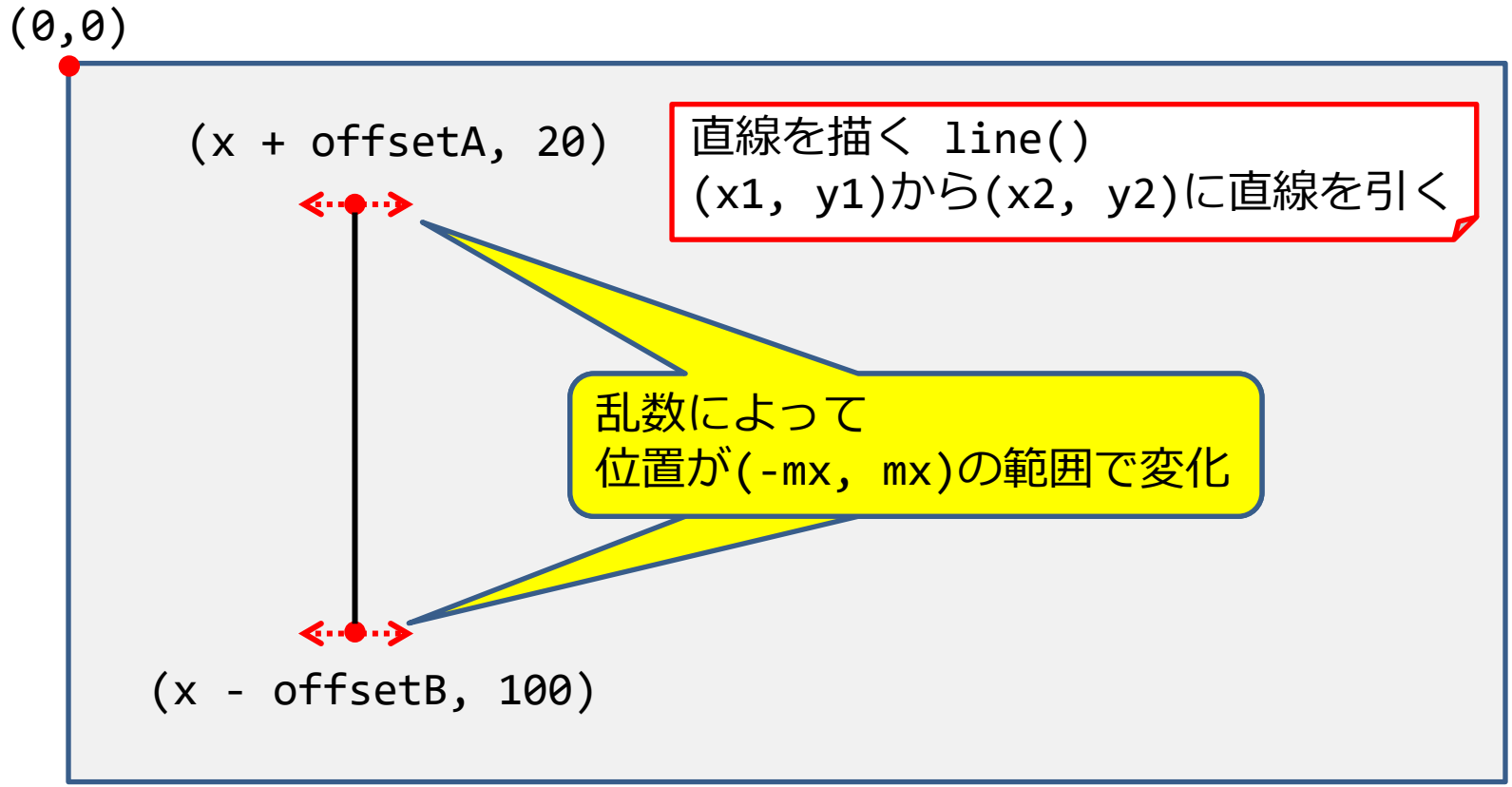
- Example 7-8 (EX_08_08)
  - ランダムに描く
  - 線の位置を乱数を使って少しずらす
- ずらす量はmouseXを使って計算

```
float mx = mouseX / 10;
float offsetA = random(-mx, mx);
float offsetB = random(-mx, mx);
line(x + offsetA, 20, x - offsetB, 100);
```

Example 7-8

```
void setup() {
  size(240, 120);
}

void draw() {
  background(204);
  for (int x = 20; x < width; x += 20) {
    float mx = mouseX / 10;
    float offsetA = random(-mx, mx);
    float offsetB = random(-mx, mx);
    line(x + offsetA, 20, x - offsetB, 100);
  }
}
```

線

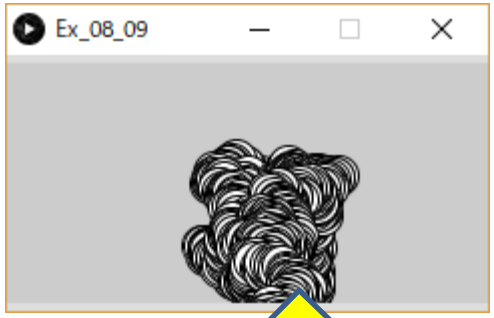

(0,0)
(x + offsetA, 20)
直線を描く line()
(x1, y1)から(x2, y2)に直線を引く
乱数によって
位置が(-mx, mx)の範囲で変化
(x - offsetB, 100)

乱数

- Example 7-9 (EX_08_09)
  - x方向・y方向にランダムで移動する
    - P102 NOTE
      同じシーケンスを生成するように強制したい場合は
      randomSeed()を使う
  - 値を制限する constrain() P203
    constrain(a, min, max)
    aの値をminとmaxの間に制限する

Example 7-9

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```



円が画面外に出ないように
constrain()でxとyを制限する

タイマー Example 7-10

- Example 7-10 (EX_08_10) 時間の経過
  - 経過時間 millis()
    - 実行開始から経過した時間をミリ秒単位で取得する

```
void draw() {
  int timer = millis();        // 経過時間を取得し，変数timerに代入
  println(timer);              // 経過時間を表示
}
```

タイマー

- Example 7-11 (EX_08_11)
  - 時限式イベント
  - millis()とif文を使って時限式イベント
    - 現在の経過時間を保存 `int currentTime = millis();`
    - イベントの設定時間と現在の経過時間を比較

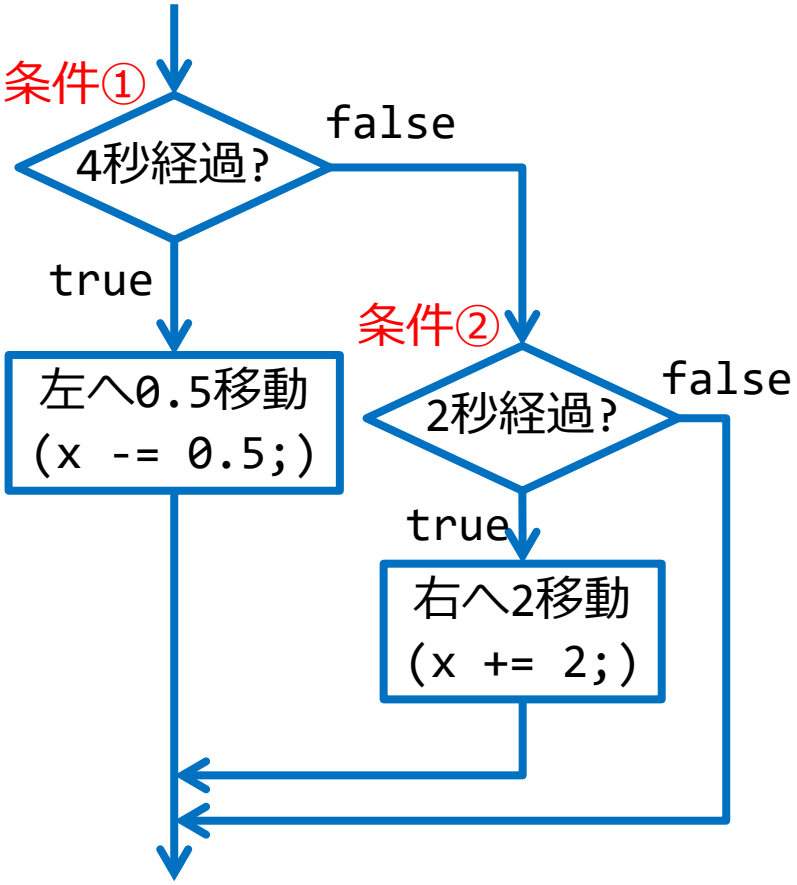
```
if(currentTime > time2){
  ~ 省略 ~
} else if(currentTime > time1){
  ~ 省略 ~
}
```

Example 7-11

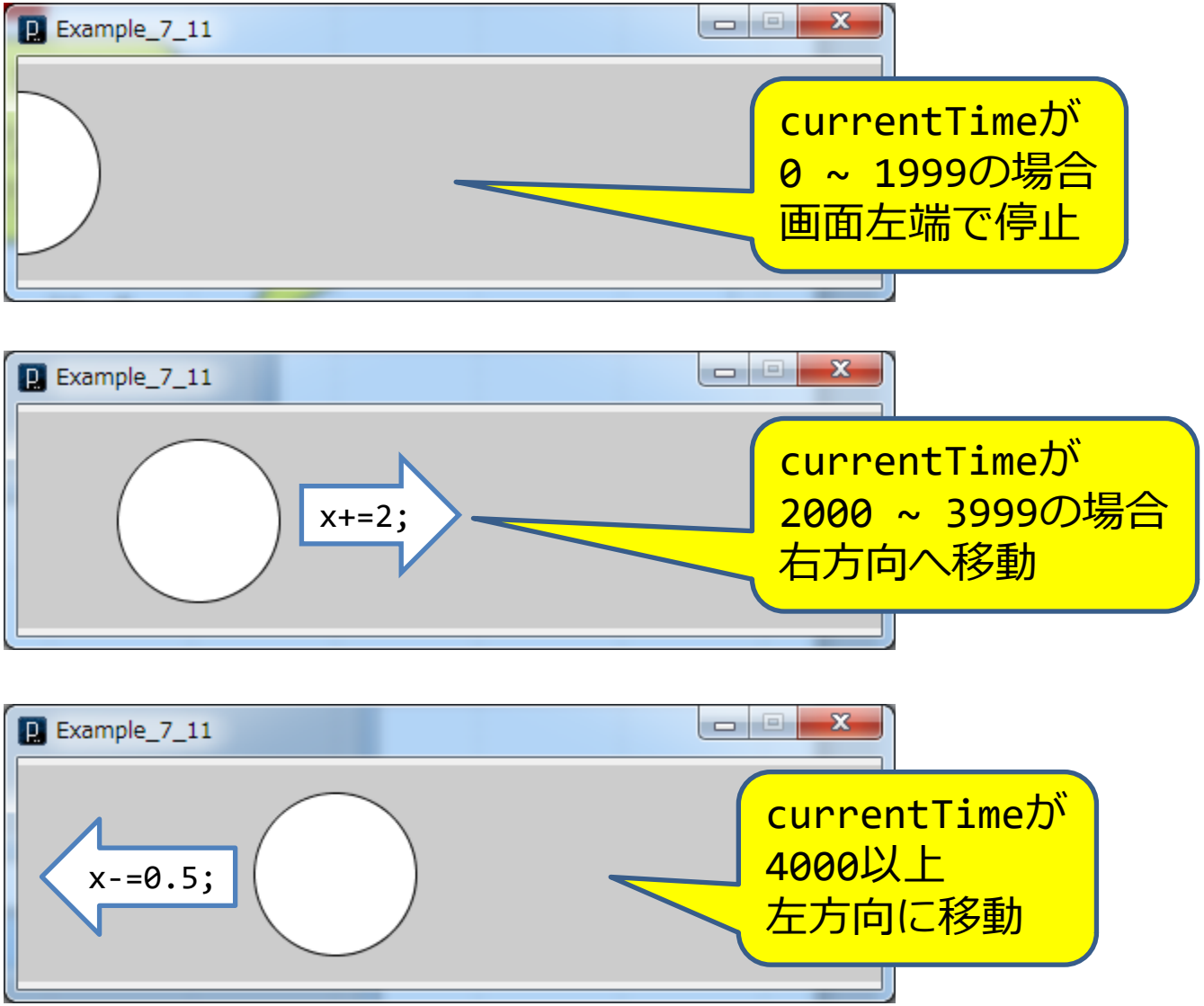
```
int time1 = 2000;            // イベント1の設定時間（ミリ秒）
int time2 = 4000;            // イベント2の設定時間（ミリ秒）
float x = 0;                 // 図形のx座標

void setup() {
  size(480, 120);
}

void draw() {
  int currentTime = millis();          // 現在の経過時間を保存する
  background(204);
  if (currentTime > time2) {           // 現在の経過時間がイベント2の時間より大きい場合
    x -= 0.5;                          // xを0.5減らす（左に0.5移動）
  } else if (currentTime > time1) {    // 条件1が偽で、イベント1の時間より大きい場合
    x += 2;                            // xを2増やす（右に2移動）
  }
  ellipse(x, 60, 90, 90);
}
```

フローチャート

- 条件の順序
  - 条件①
    経過時間が4000ミリ秒より大きい（4秒経過）
  - 条件②
    条件①が偽で、
    経過時間が2000ミリ秒より大きい（2秒経過）



実行結果


Example_7_11
currentTimeが
0 ~ 1999の場合
画面左端で停止
Example_7_11
x+=2;
currentTimeが
2000 ~ 3999の場合
右方向へ移動
Example_7_11
x-=0.5;
currentTimeが
4000以上
左方向に移動

円運動

- Example 7-12 (EX_08_12)
  - 三角関数 sin() P204
  - sin()の値は-1から+1の範囲
  - map()を使って(-1,1)を(0,255)の範囲に変換
  - 変換後の(0,255)の値で背景色を設定
    黒～灰～白の間を変化する

Example 7-12

```
float angle = 0.0;

void draw() {
  float sinval = sin(angle);
  println(sinval);
  float gray = map(sinval, -1, 1, 0, 255);
  background(gray);
  angle += 0.1;
}
```

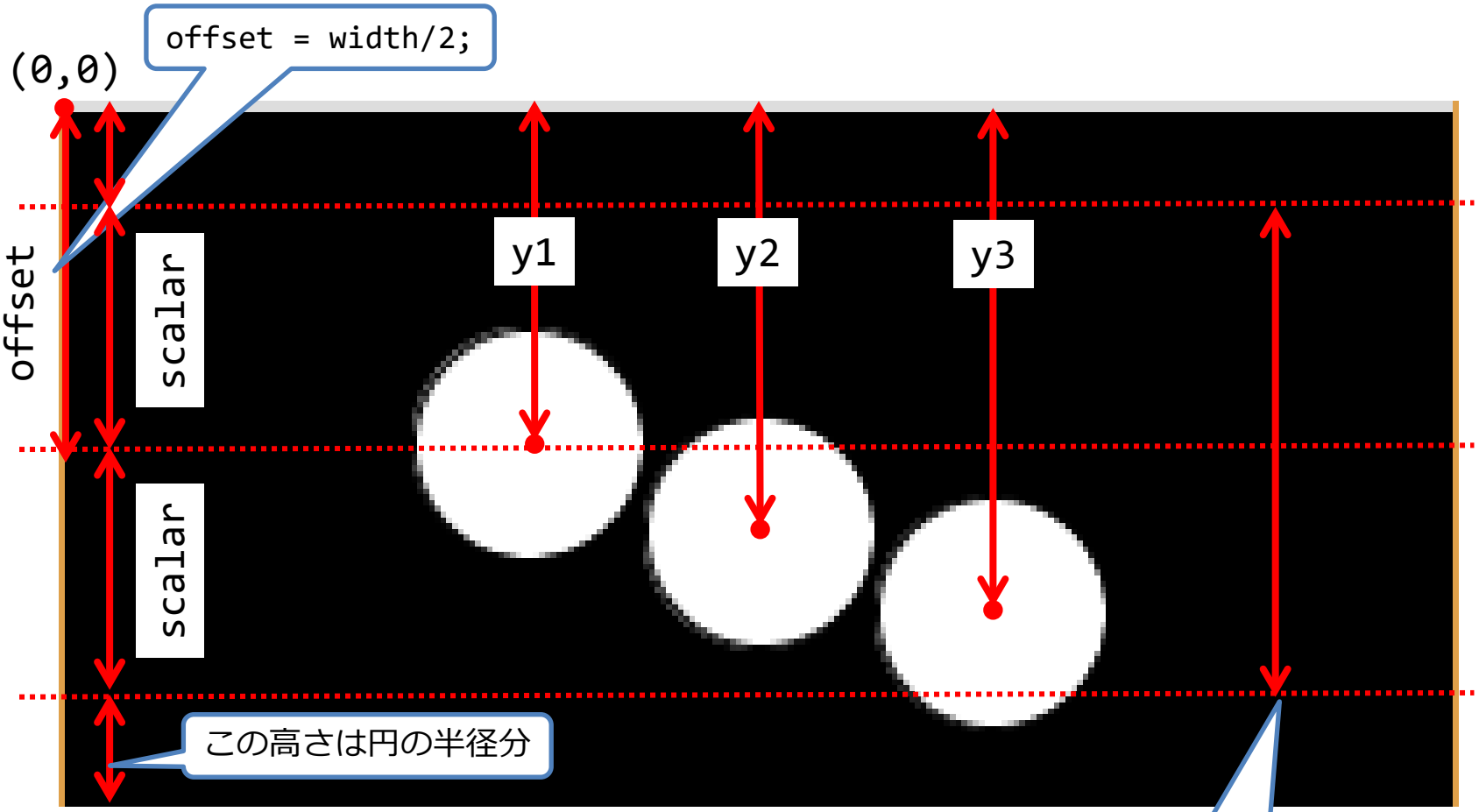
円運動

- Example 7-13 (EX_08_13)
  - sin()の値を動きに変換
  - 角度angle, sin()の値と円運動の関係は教科書P104図7-2を参照

Example 7-13

```
float angle = 0.0;
float offset = 60;
float scalar = 40;
float speed = 0.05;

void setup() {
  size(240, 120);
}

void draw() {
  background(0);
  float y1 = offset + sin(angle) * scalar;
  float y2 = offset + sin(angle + 0.4) * scalar;
  float y3 = offset + sin(angle + 0.8) * scalar;
  ellipse( 80, y1, 40, 40);
  ellipse(120, y2, 40, 40);
  ellipse(160, y3, 40, 40);
  angle += speed;
}
```

実行結果




```
y1 = offset + sin(angle) * scalar;
y2 = offset + sin(angle + 0.4) * scalar;
y3 = offset + sin(angle + 0.8) * scalar;
```

円運動

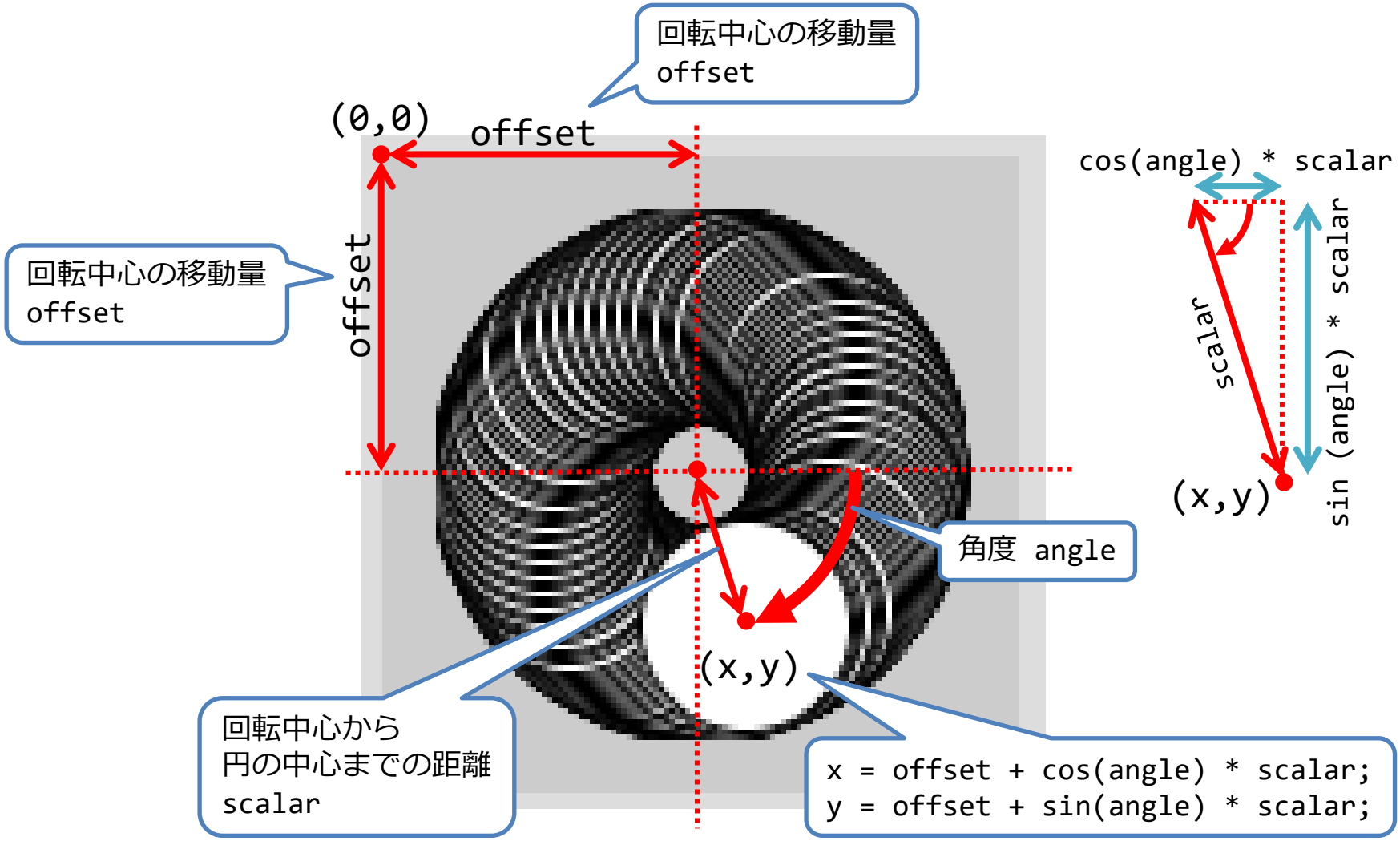
- Example 7-14 (EX_08_14) 円運動
  - sin()とcos()を使って円運動させる
  - 変数offsetで回転の中心を(0,0)から(offset, offset)に移す
  - 変数scalarは回転中心から円の中心までの距離
  - 変数angleは回転の角度
- (x,y)をcos()とsin()で計算

```
float x = offset + cos(angle) * scalar;
float y = offset + sin(angle) * scalar;
```

Example 7-14

```
float angle = 0.0;
float offset = 60;
float scalar = 30;
float speed = 0.05;

void setup() {
  size(120, 120);
}

void draw() {
  float x = offset + cos(angle) * scalar;
  float y = offset + sin(angle) * scalar;
  ellipse( x, y, 40, 40);
  angle += speed;
}
```

実行結果


回転中心の移動量
offset
(0,0)
offset
回転中心の移動量
offset
offset
cos(angle) * scalar
scalar
sin (angle) * scalar
(x,y)
角度 angle
(x,y)
回転中心から
円の中心までの距離
scalar
x = offset + cos(angle) * scalar;
y = offset + sin(angle) * scalar;

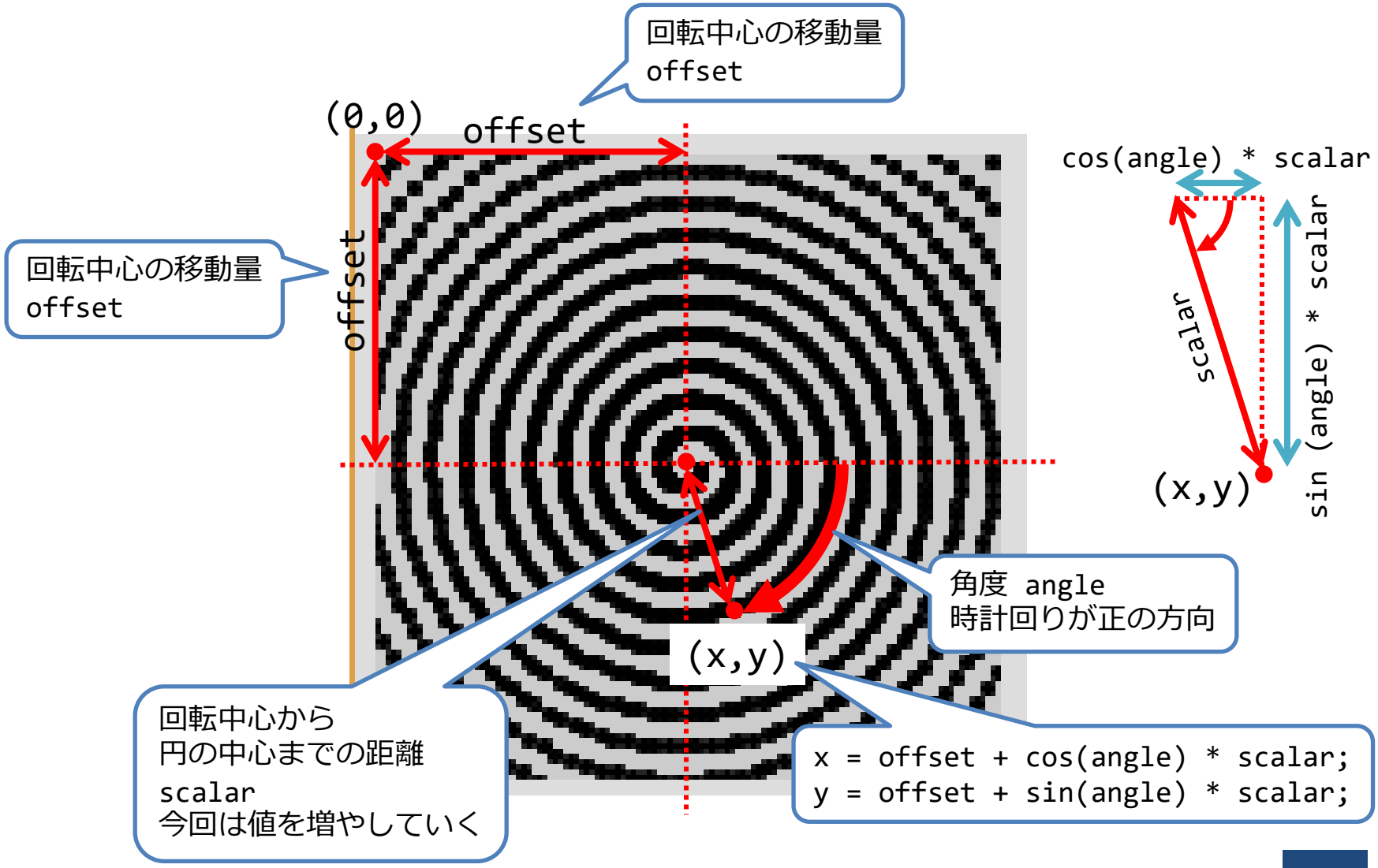
円運動

- Example 7-15 (EX_08_15) らせん
  - Example 7-14 とほぼ同じ
  - scalarを増やす⇒
    回転中心から円の中心までの距離が延びる⇒
    らせんを描く

Example 7-15

```
float angle = 0.0;
float offset = 60;
float scalar = 2;
float speed = 0.05;

void setup() {
  size(120, 120);
  fill(0);
}

void draw() {
  float x = offset + cos(angle) * scalar;
  float y = offset + sin(angle) * scalar;
  ellipse( x, y, 2, 2);
  angle += speed;
  scalar += speed;       // 回転中心から円の中心までの距離を増やす
}
```

実行結果


回転中心の移動量
offset
(0,0)
offset
回転中心の移動量
offset
offset
cos(angle) * scalar
scalar
sin (angle) * scalar
(x,y)
角度 angle
時計回りが正の方向
(x,y)
回転中心から
円の中心までの距離
scalar
今回は値を増やしていく
x = offset + cos(angle) * scalar;
y = offset + sin(angle) * scalar;